HandsMen Threads Salesforce Project Documentation

Prepared for Project Review

Salesforce Platform Developer Project

Data Model Automation Security Apex Testing

Contents

Section	Description
1	Project Overview
2	Objectives and Implementation Scope
3	Requirement Analysis and Data Model
4	Salesforce Configuration and Data Security
5	Data Quality and Business Rules
6	Email Templates and Process Automation
7	Apex Triggers and Handler Logic
8	Batch Apex and Scheduled Processing
9	User Interface and Sample Records
10	Functional Testing Evidence
11	Conclusion

This report presents the HandsMen Threads Salesforce implementation in a concise review format. Each numbered figure panel identifies the exact screenshot that should be captured from the Salesforce developer org before submission.

1 Project Overview

HandsMen Threads is a Salesforce solution for a fashion business that manages customers, products, orders, inventory, and marketing campaigns. The project brings operational data into one platform and applies automation to improve customer communication, stock control, loyalty management, and order processing.

The implementation combines custom objects, custom fields, lookup and master detail relationships, formula fields, validation rules, profiles, roles, permission sets, email templates, record triggered flows, scheduled automation, Apex triggers, Batch Apex, and Scheduled Apex.

Main Business Outcomes

- Send an automatic confirmation message when an order is confirmed.
- Update customer loyalty status from purchase history.
- Notify the inventory team when stock falls below the defined threshold.
- Calculate order totals and deduct stock when confirmed orders are processed.
- Process inventory updates in bulk through scheduled Apex.

Technology Used

Area	Salesforce Feature
Data	Custom objects, custom fields, formula fields, relationships
Automation	Record triggered flows and scheduled flow
Development	Apex classes, triggers, handler pattern, Batch Apex, Scheduled Apex
Security	Profiles, roles, users, permission sets, object access
Communication	Classic email templates and email alerts

2 Objectives and Implementation Scope

2.1 Project Objectives

- Create a structured Salesforce data model for customers, products, orders, inventory, and campaigns.
- Maintain accurate customer, order, and stock information through UI level validation.
- Automate order confirmation, low stock alerts, and loyalty status updates.
- Apply role based access so teams can work only with relevant business records.
- Use Apex for order validation, total calculation, stock deduction, and bulk processing.

2.2 Current Implementation Scope

Area	Included Implementation	Review Evidence
Data model	Five custom objects with business fields and relationships	Object Manager and Schema Builder
Data quality	Formula fields and validation rules	Field and validation rule screens
Security	Profile, role hierarchy, users, permission set	Setup security screens
Automation	Order, stock, and loyalty flows	Flow Builder canvases
Development	Handler, triggers, batch, and scheduler	Apex class and trigger screens
Testing	Sample records, error messages, updates, and emails	Lightning records and test results

3 Requirement Analysis and Data Model

The data model separates master data from transaction data. Customer and product records store reusable business information. Orders capture transactions, inventory records track stock by product, and marketing campaigns connect promotional activity with customers.

Custom Object	Key Fields	Business Purpose
HandsMen Customer	Name, First Name, Last Name, Email, Phone, Loyalty Status, Total Purchases	Stores customer identity, contact, and loyalty details.
HandsMen Product	Name, SKU, Price, Stock Quantity	Stores the product catalogue and pricing.
HandsMen Order	Order Number, Status, Quantity, Total Amount, Customer, Product	Captures customer purchases and order status.
Inventory	Inventory Number, Warehouse, Stock Quantity, Product	Tracks stock levels for each product.
Marketing Campaign	Campaign Number, Start Date, End Date, Customer	Stores promotional campaigns and customer association.

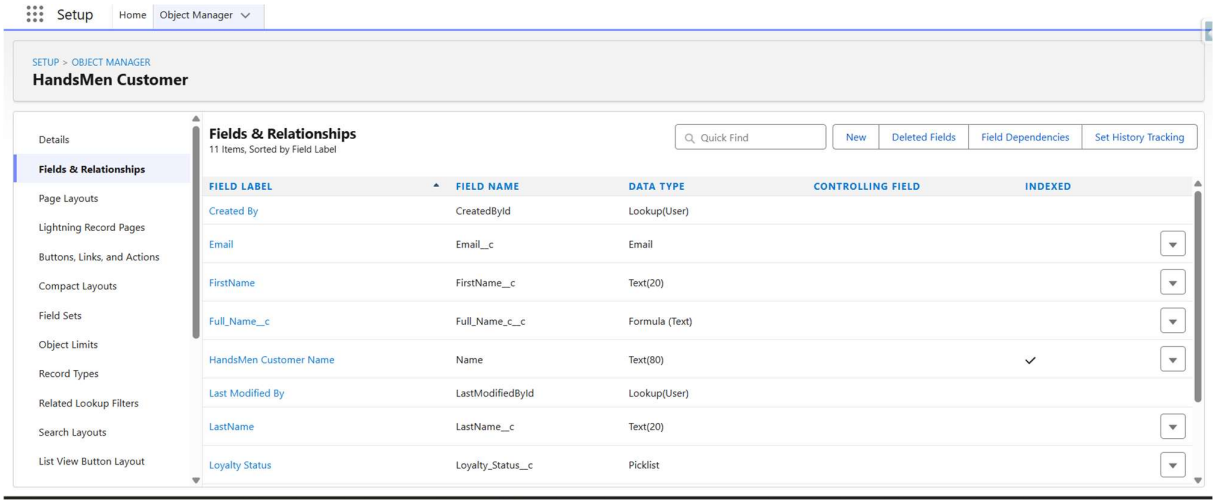


Figure 1 HandsMen Customer object fields including email phone loyalty status and total purchases

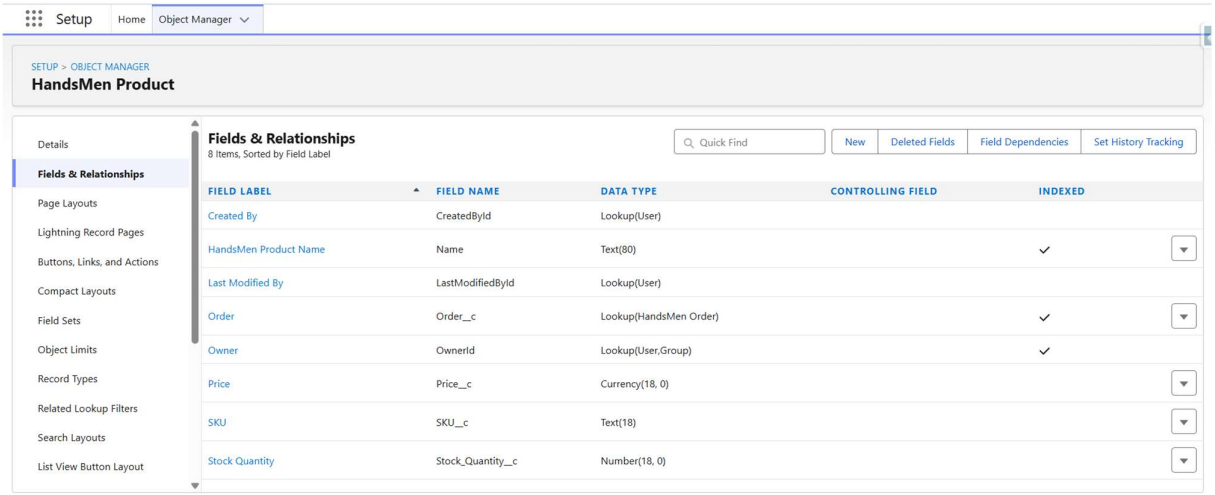


Figure 2 HandsMen Product object fields including SKU price and stock quantity

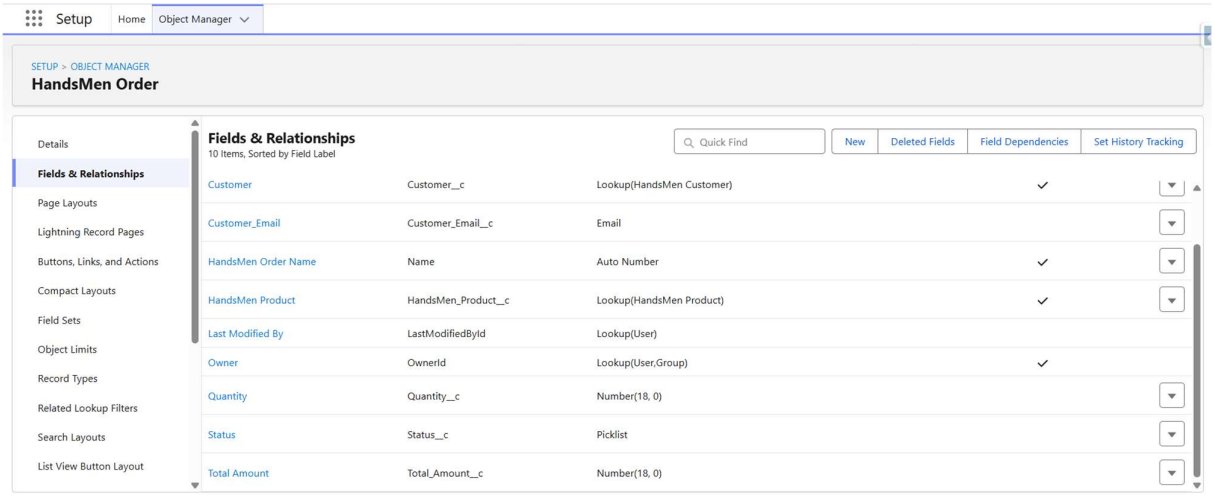


Figure 3 HandsMen Order object fields and relationships including status quantity total amount customer and product

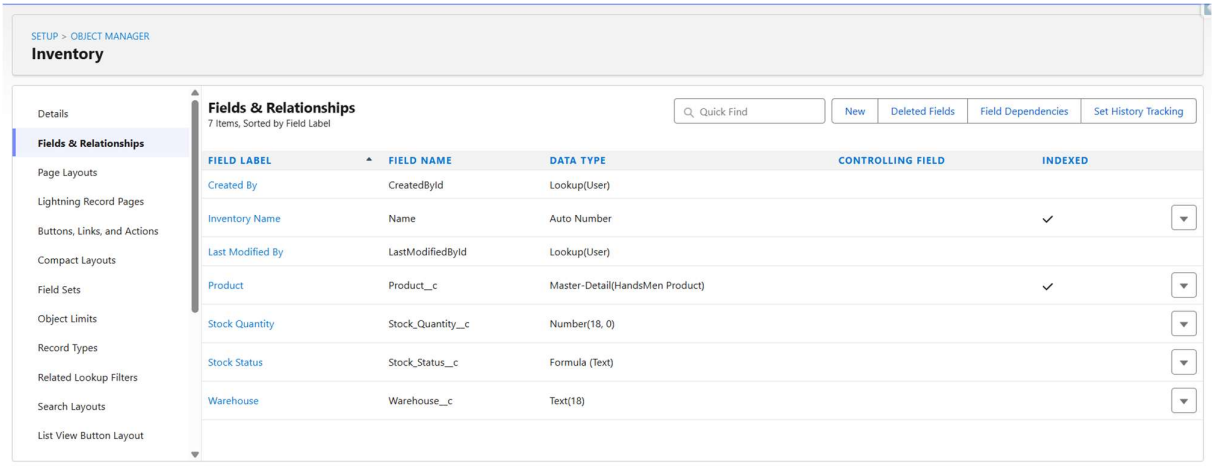


Figure 4 Inventory object fields and relationship including warehouse stock quantity and related product

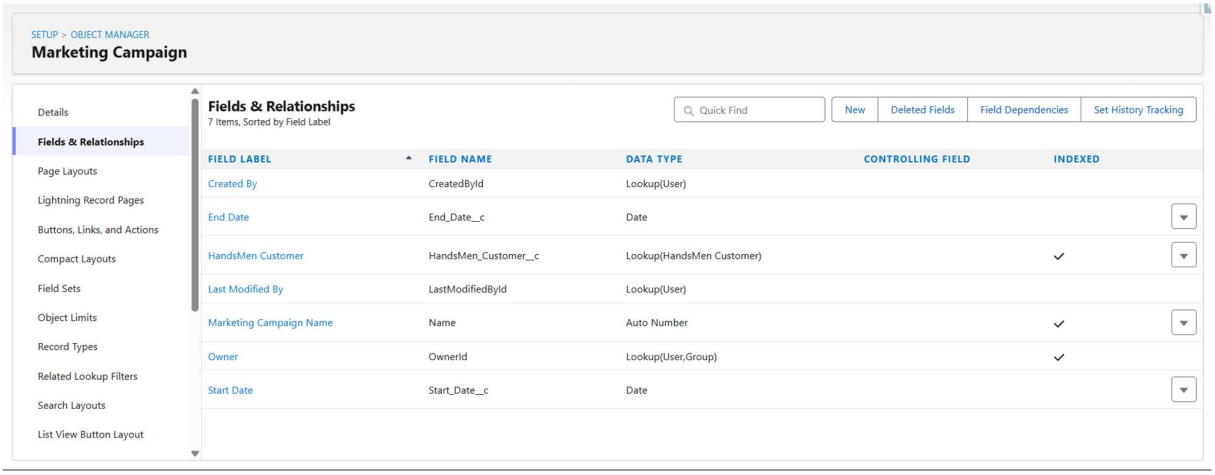


Figure 6 Marketing Campaign object fields and relationship including campaign dates and related customer

3.1 Relationships and Important Fields

Child Object	Relationship	Related Object	Purpose
Marketing Campaign	Lookup	HandsMen Customer	Associates a campaign with a customer.
HandsMen Product	Lookup	HandsMen Order	Connects product information to an order as documented.
HandsMen Order	Lookup	HandsMen Customer	Links each order to the purchasing customer.
Inventory	Master Detail	HandsMen Product	Makes inventory dependent on its product.

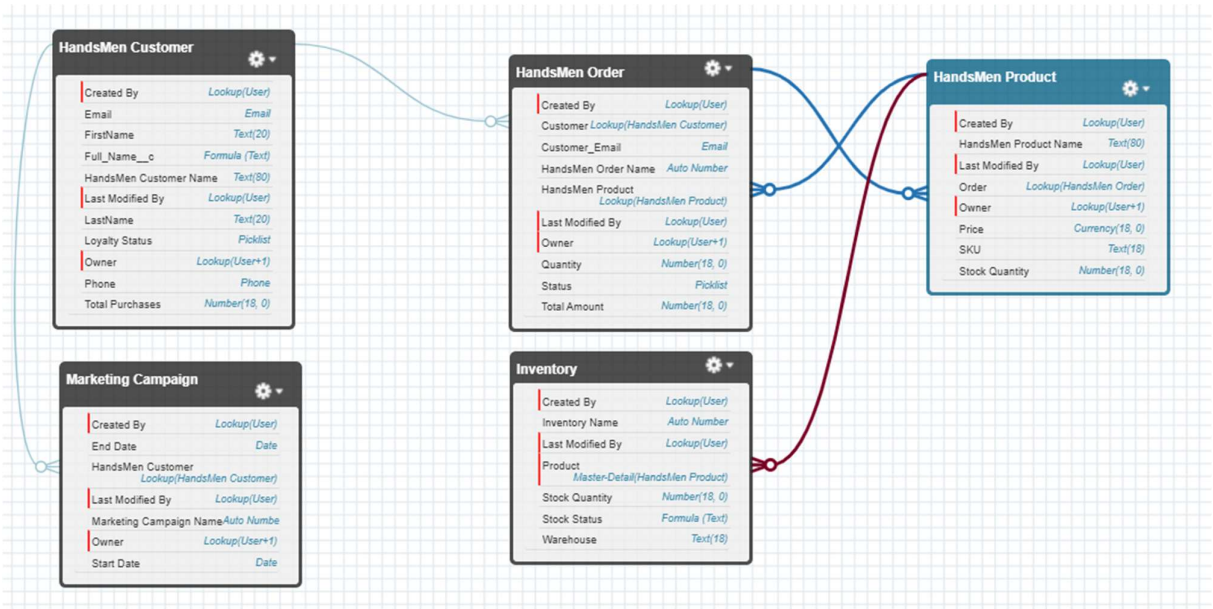


Figure 2 Schema Builder Data Model

4 Salesforce Configuration and Data Security

The project uses custom tabs and a dedicated navigation workspace so users can open customer, product, order, inventory, and campaign records from Lightning Experience. Security is controlled through profiles, roles, and a permission set.

Role	Intended Access
Sales Manager	Full access to customer and order records.
Inventory Manager	Read and edit access to inventory and product records.
Marketing Manager or Team	Read customer records and manage marketing campaign records.

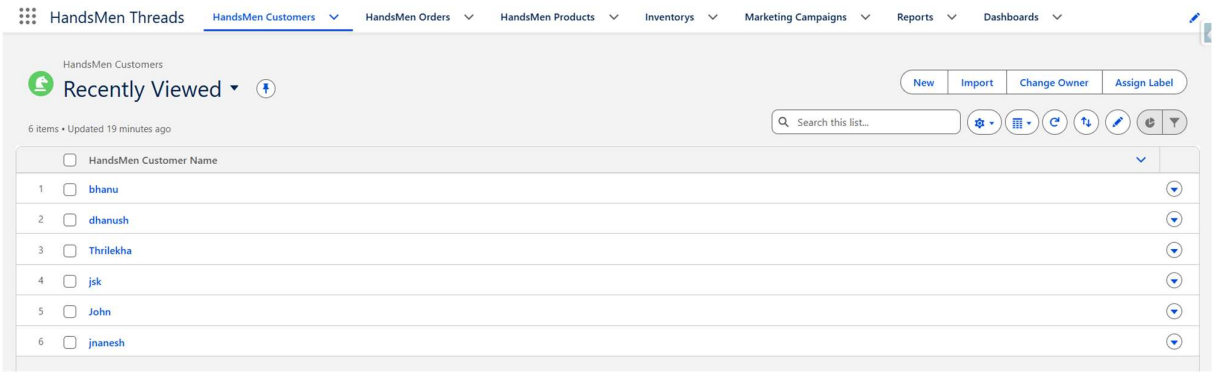


Figure 4 HandsMen Lightning App and Navigation



Figure 5 Role Hierarchy

4.1 Profiles Users and Permission Sets

Security Item	Configuration	Purpose
Profile	Platform 1 cloned from Standard User	Provides the base Salesforce Platform access.
User	Sales user assigned to Sales role	Supports customer and order activities.
User	Inventory user assigned to Inventory role	Supports product and inventory activities.
Permission Set	Permission Platform 1	Adds create, read, edit, and delete access for selected project objects.

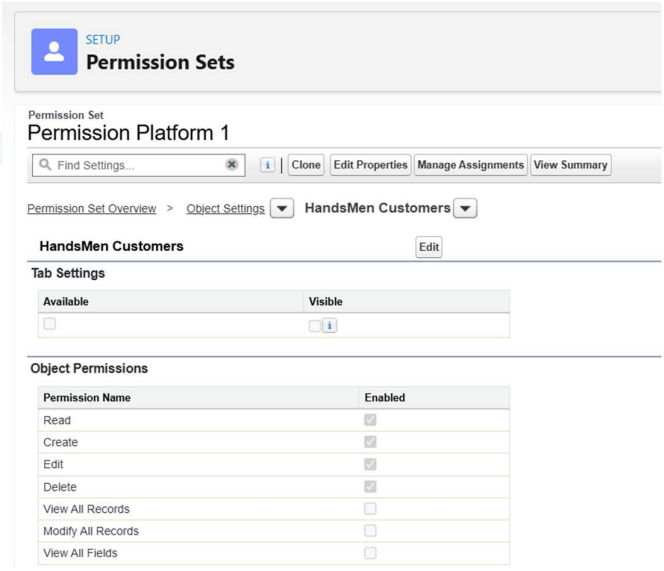


Figure 1 HandsMen Customer permissions configured in Permission Platform 1

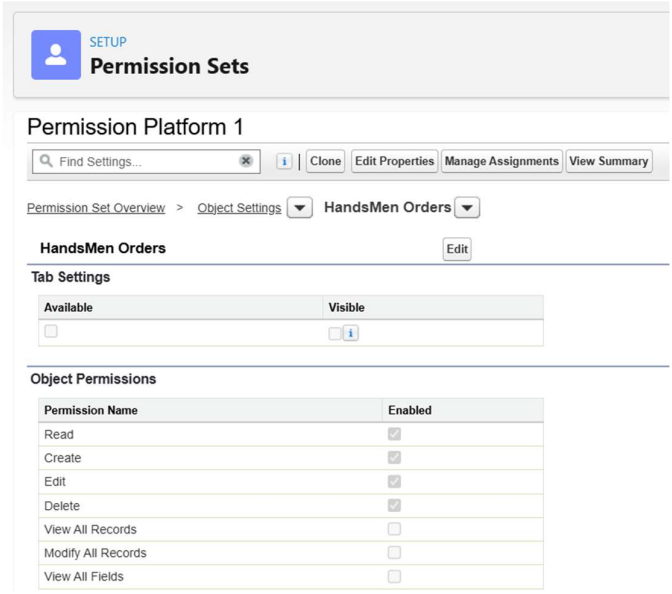


Figure 2 HandsMen Order permissions configured in Permission Platform 1

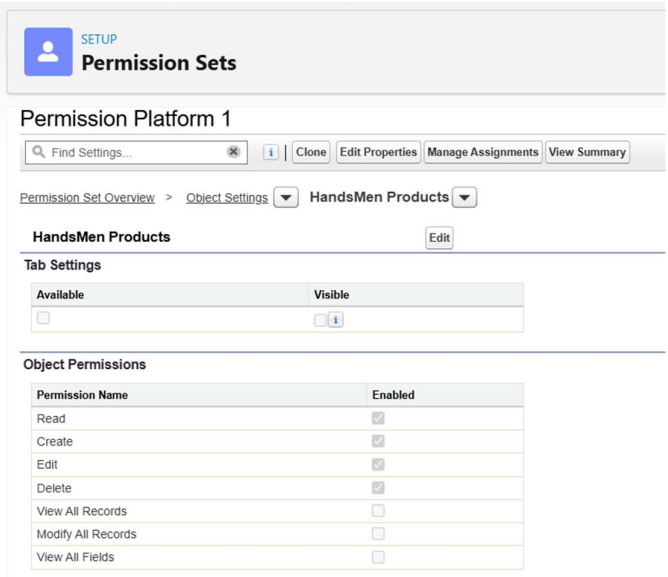


Figure 3 HandsMen Products permissions configured in Permission Platform 1

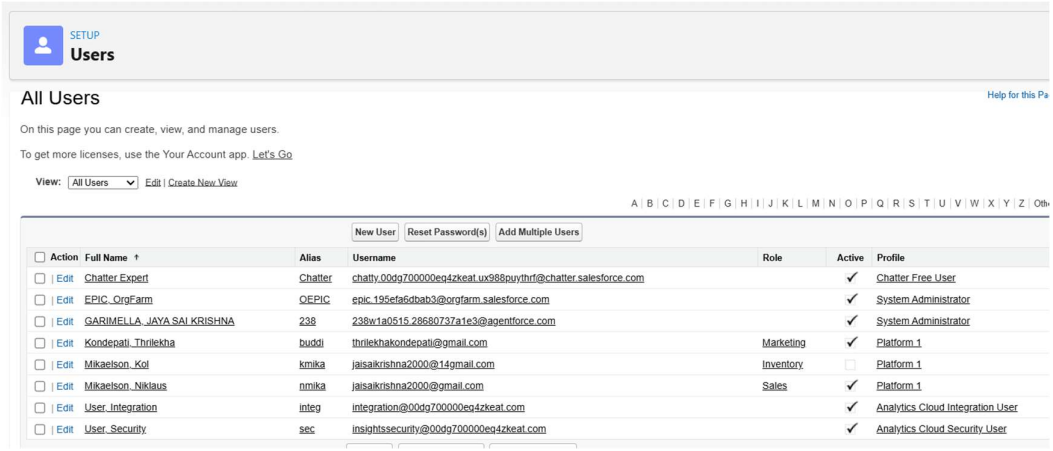


Figure 4 Project Users and Assigned Roles

5 Data Quality and Business Rules

5.1 Formula Fields

Object	Formula Field	Return Type	Formula
HandsMen Customer	Full Name	Text	FirstName__c + " " + LastName__c
Inventory	Stock Status	Text	IF(Stock_Quantity__c > 10, "Available", "Low Stock")

5.2 Validation Rules

Object	Rule	Condition	Expected Message
HandsMen Order	Total Amount	Total_Amount__c <= 0	Please enter a correct amount.
Inventory	Stock Quantity	Stock_Quantity__c <= 0	Inventory count cannot be less than or equal to zero.
HandsMen Customer	Email	NOT(CONTAINS(Email__c, "@gmail.com"))	Please enter a correct Gmail address.

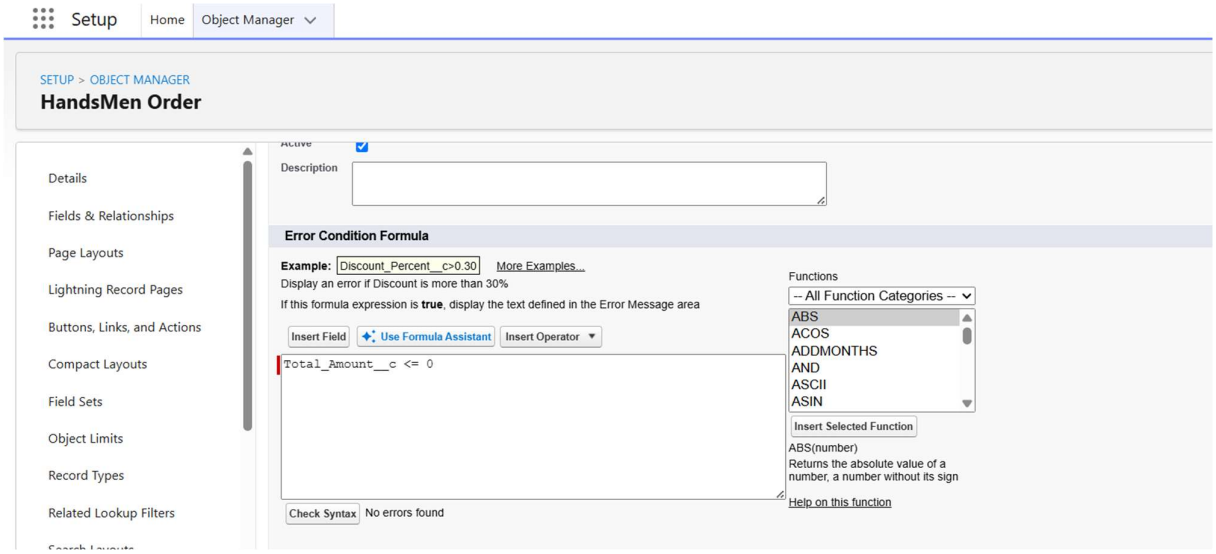


Figure 1 Total Amount field configuration and successful syntax validation

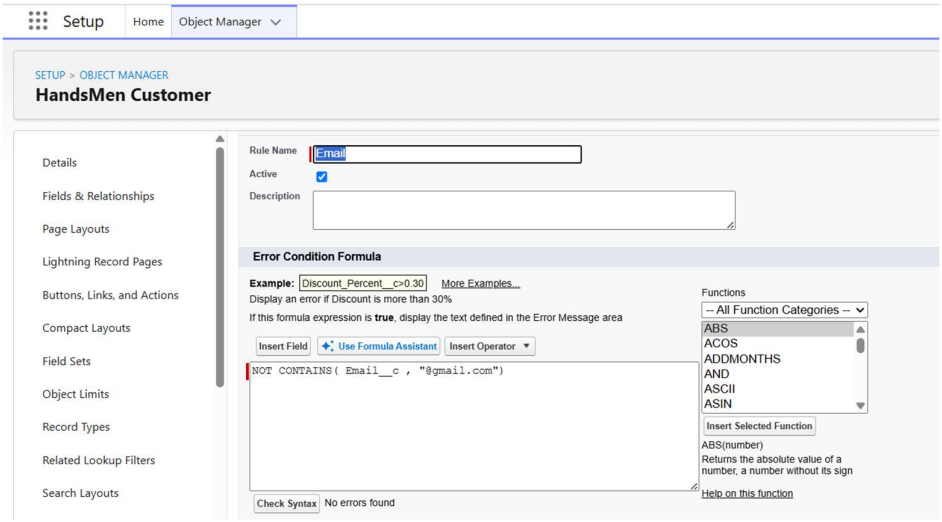


Figure 2 Email formula field configuration and successful syntax validation

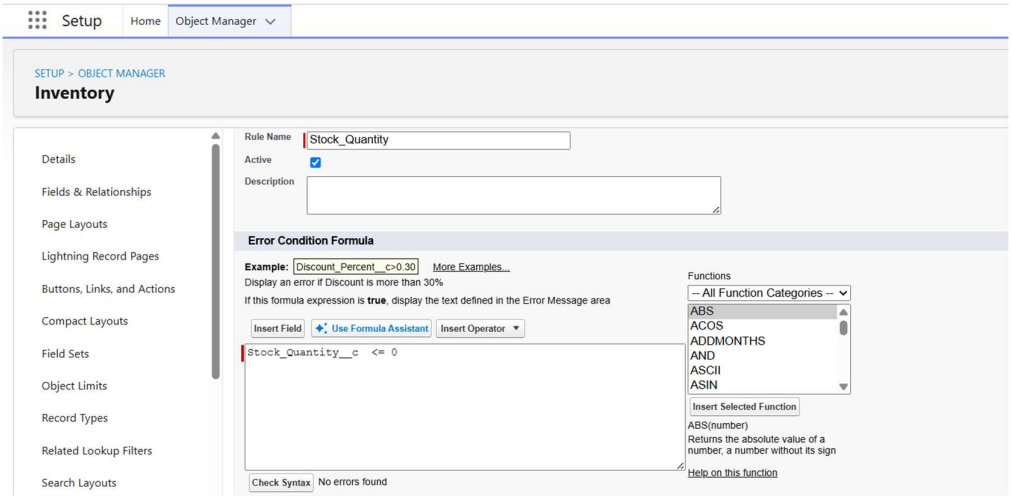


Figure 3 Stock Quantity configuration and successful syntax validation

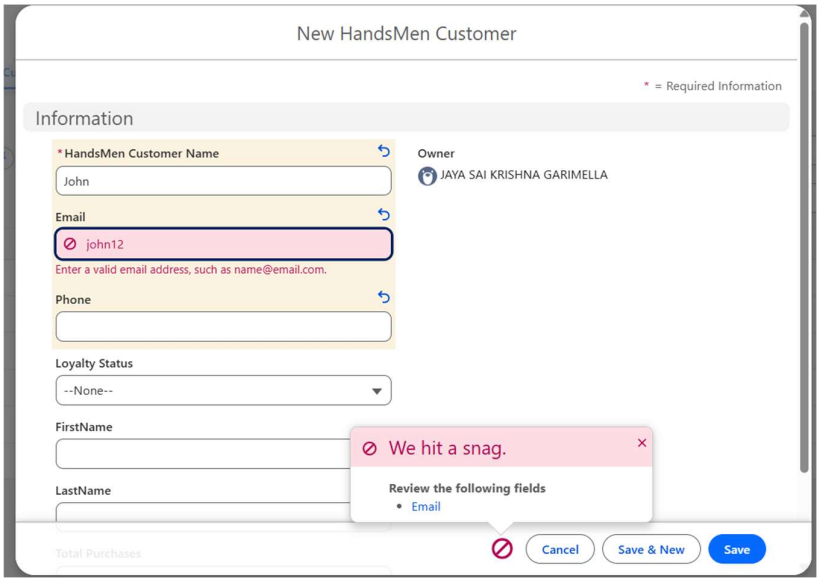


Figure 3 Customer email validation rule preventing an invalid email from being saved

6 Email Templates and Process Automation

6.1 Email Templates and Alerts

Template	Type	Used For
Order Confirmation	HTML	Sent when an order changes to Confirmed.
Low Stock Alert	Text	Sent to the inventory team when stock is below five.
Loyalty Program Email	HTML	Supports customer loyalty communication.

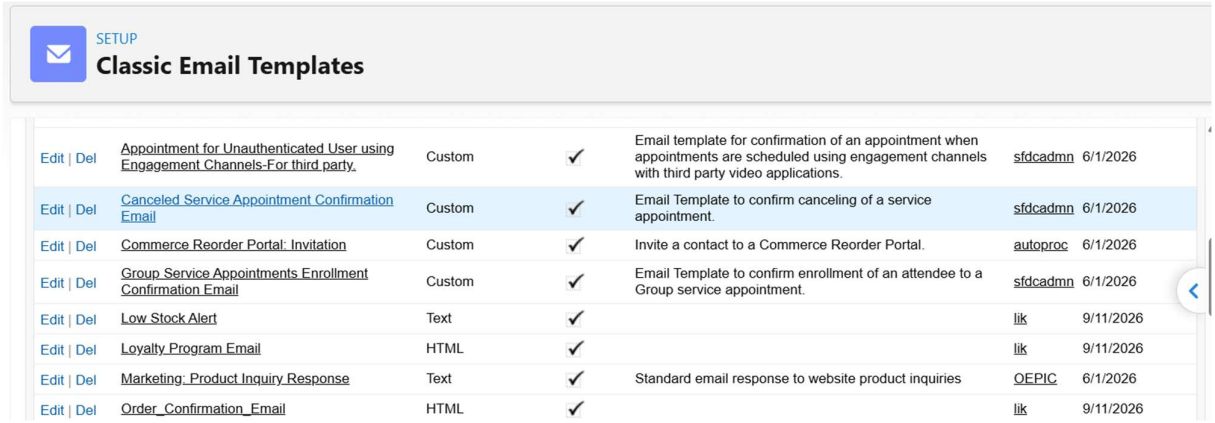


Figure 10 Email Templates List

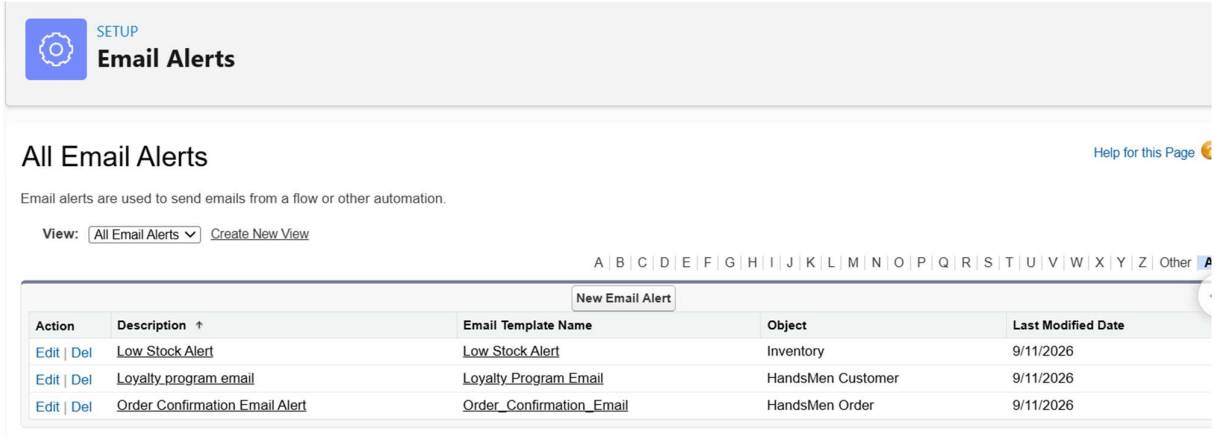


Figure 11 Order Confirmation Email Alert

6.2 Salesforce Flows

Flow	Type	Start Condition	Action
Order Confirmation Flow	Record Triggered	Order status changes to Confirmed	Sends the order confirmation email alert.
Stock Alert Flow	Record Triggered	Inventory stock quantity is below 5	Sends a low stock email alert.
Loyalty Status Update Flow	Schedule Triggered	Runs daily	Sets Gold, Silver, or Bronze from Total Purchases.

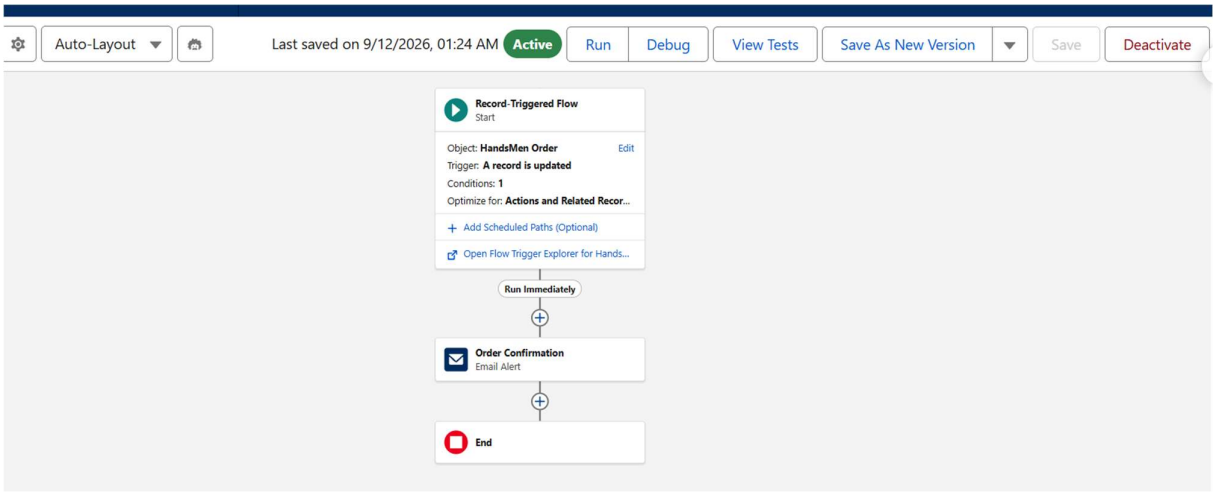


Figure 12 Order Confirmation Flow

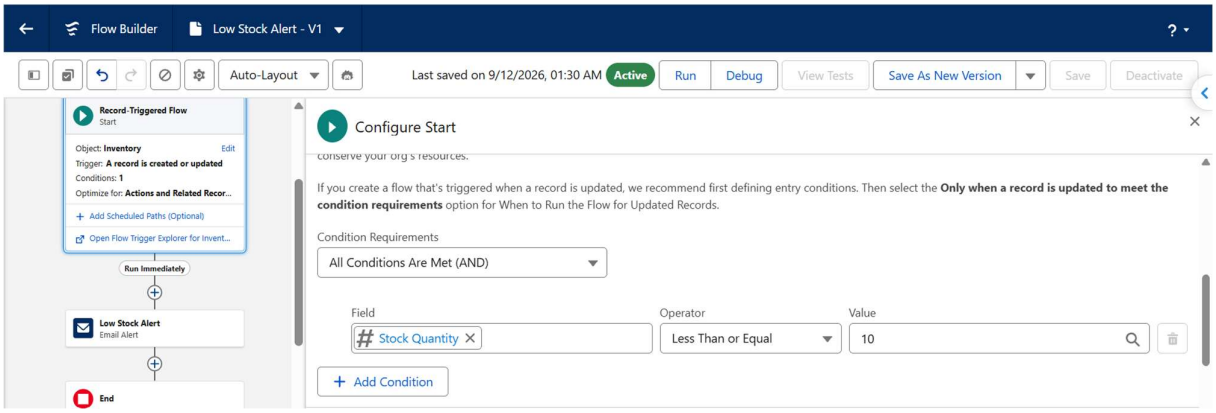


Figure 13 Stock Alert Flow

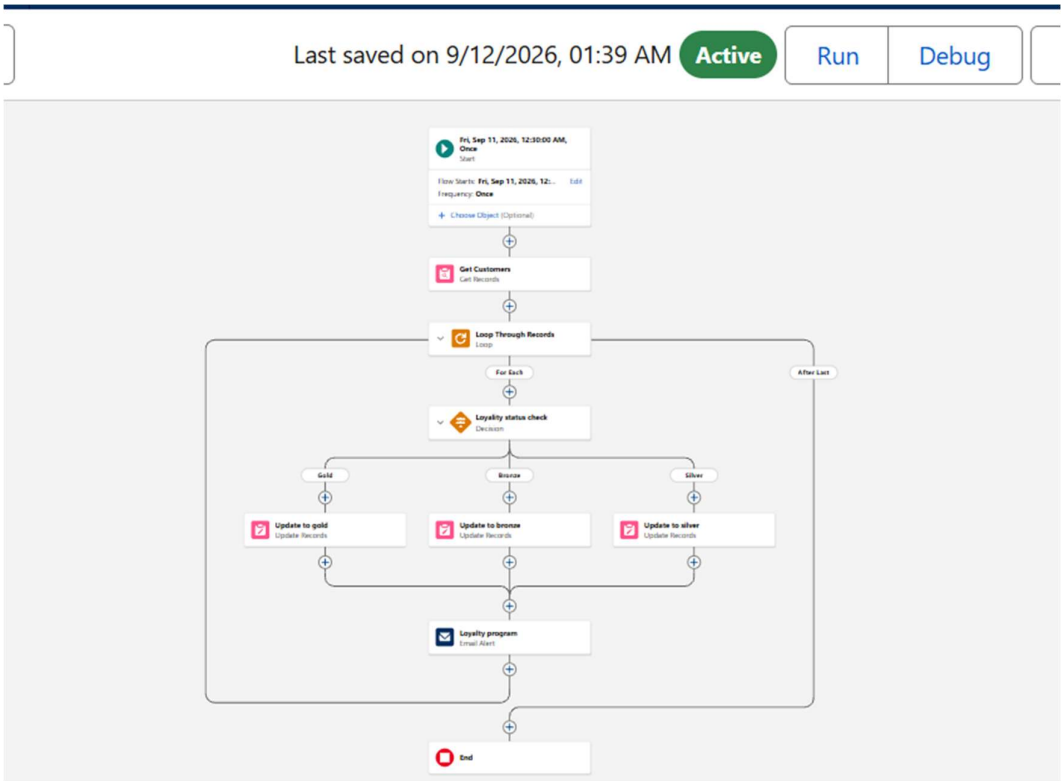


Figure 14 Loyalty Status Update Flow

7 Apex Triggers and Handler Logic

Apex is used where record processing requires reusable logic and transaction level enforcement. The order trigger delegates status based quantity validation to OrderTriggerHandler so the trigger remains small and the rule can be maintained in one class.

Status	Quantity Rule	Outcome
Confirmed	Quantity must be greater than 500	The record is blocked when the quantity is missing or too low.
Pending	Quantity must be greater than 200	The record is blocked when the quantity is missing or too low.
Rejection	Quantity must equal 0	Rejected orders cannot contain a positive quantity.

```
trigger OrderTrigger on HandsMen_Order__c (before insert, before update) {
    OrderTriggerHandler.validateOrderQuantity(Trigger.new);
}
```

```
public class OrderTriggerHandler {

    public static void validateOrderQuantity(List<HandsMen_Order__c> orderList) {

        for (HandsMen_Order__c order : orderList) {

            if (order.Status__c == 'Confirmed') {

                if (order.Quantity__c == null || order.Quantity__c <= 500) {

                    order.Quantity__c.addError('For Status "Confirmed", Quantity must be more than 500.');

                }

            } else if (order.Status__c == 'Pending') {

                if (order.Quantity__c == null || order.Quantity__c <= 200) {

                    order.Quantity__c.addError('For Status "Pending", Quantity must be more than 200.');

                }

            } else if (order.Status__c == 'Rejection') {

                if (order.Quantity__c == null || order.Quantity__c != 0) {

                    order.Quantity__c.addError('For Status "Rejection", Quantity must be 0.');

                }

            }

        }
```

Figure 15 OrderTriggerHandler Apex Class

```
trigger OrderTrigger on HandsMen_Order__c (before insert, before update) {

    if (Trigger.isBefore && (Trigger.isInsert || Trigger.isUpdate)) {

        OrderTriggerHandler.validateOrderQuantity(Trigger.new);

    }

}
```

Figure 16 OrderTrigger Apex Trigger

7.1 Order Total and Stock Deduction Triggers

Apex Artifact	Event	Responsibility
Order Total Trigger	Before insert and before update	Reads the related product price and sets Total Amount to Quantity multiplied by Price.
Stock Deduction Trigger	After insert and after update	When an order becomes Confirmed, totals ordered quantity by product and reduces Inventory Stock Quantity.

The stock deduction design collects product identifiers in a set, aggregates quantities in a map, queries matching inventory records once, and performs one bulk update. This avoids database operations inside the record loop.

```
trigger OrderTotalTrigger on HandsMen_Order__c (before insert, before update) {
    Set<Id> productIds = new Set<Id>();

    for (HandsMen_Order__c order : Trigger.new) {
        if (order.HandsMen_Product__c != null) {
            productIds.add(order.HandsMen_Product__c);
        }
    }

    Map<Id, HandsMen_Product__c> productMap = new Map<Id, HandsMen_Product__c>(
        [SELECT Id, Price__c FROM HandsMen_Product__c WHERE Id IN :productIds]
    );

    for (HandsMen_Order__c order : Trigger.new) {
        if (order.HandsMen_Product__c != null && productMap.containsKey(order.HandsMen_Product__c)) {
            HandsMen_Product__c product = productMap.get(order.HandsMen_Product__c);
            if (order.Quantity__c != null) {
                order.Total_Amount__c = order.Quantity__c * product.Price__c;
            }
        }
    }
}
```

Figure 17 Order Total Trigger

```
trigger StockDeductionTrigger on HandsMen_Order__c (after insert, after update) {
    Set<Id> productIds = new Set<Id>();
    for (HandsMen_Order__c order : Trigger.new) {
        Boolean shouldDeduct = false;

        if (order.Status__c == 'Confirmed') {

            if (Trigger.isInsert) {
                shouldDeduct = true;
            }
            else if (Trigger.isUpdate) {
                HandsMen_Order__c oldOrder = Trigger.oldMap.get(order.Id);

                if (oldOrder.Status__c != 'Confirmed') {
                    shouldDeduct = true;
                }
            }
        }

        if (shouldDeduct && order.HandsMen_Product__c != null) {
            productIds.add(order.HandsMen_Product__c);
        }
    }

    if (productIds.isEmpty()) {
        return;
    }
```

```
// Get inventory for the products
Map<Id, Inventory__c> inventoryMap = new Map<Id, Inventory__c>();

for (Inventory__c inv : [
    SELECT Id, Stock_Quantity__c, Product__c
    FROM Inventory__c
    WHERE Product__c IN :productIds
]) {
    inventoryMap.put(inv.Product__c, inv);
}

List<Inventory__c> inventoriesToUpdate = new List<Inventory__c>();

// Deduct stock
for (HandsMen_Order__c order : Trigger.new) {

    Boolean shouldDeduct = false;

    if (order.Status__c == 'Confirmed') {

        if (Trigger.isInsert) {
            shouldDeduct = true;
        }
        else if (Trigger.isUpdate) {
            HandsMen_Order__c oldOrder = Trigger.oldMap.get(order.Id);

            if (oldOrder.Status__c != 'Confirmed') {
                shouldDeduct = true;
            }
        }
    }
```

```
        if (shouldDeduct &&
            order.HandsMen_Product__c != null &&
            order.Quantity__c != null) {

            Inventory__c existingInventory =
                inventoryMap.get(order.HandsMen_Product__c);

            if (existingInventory != null) {

                Decimal currentStock =
                    existingInventory.Stock_Quantity__c == null
                    ? 0
                    : existingInventory.Stock_Quantity__c;

                Decimal newStock =
                    currentStock - order.Quantity__c;

                inventoriesToUpdate.add(
                    new Inventory__c(
                        Id = existingInventory.Id,
                        Stock_Quantity__c = newStock
                    )
                );
            }
        }
    }

    if (!inventoriesToUpdate.isEmpty()) {
        update inventoriesToUpdate;
    }
}
```

Figure 18 Stock Deduction Trigger

8 Batch Apex and Scheduled Processing

InventoryBatchJob implements Database Batchable and Schedulable. It identifies low stock products, applies the restocking logic in manageable batches, and can be scheduled for automatic execution.

Method or Process	Responsibility	Expected Evidence
start	Queries products that meet the low stock condition.	SOQL query and selected fields.
execute	Adds the restock quantity and updates records in bulk.	Loop, update list, and DML statement.
finish	Records that inventory synchronization completed.	Completion debug statement.
Schedulable execute	Starts the batch with a scope of 200.	Database executeBatch call.
Scheduled job	Runs Daily Inventory Sync automatically.	Scheduled Jobs entry and next run time.

```
global class InventoryBatchJob implements Database.Batchable<SObject>, Schedulable {

global Database.QueryLocator start(Database.BatchableContext BC) {

return Database.getQueryLocator(

'SELECT Id, Stock_Quantity__c FROM Product__c WHERE Stock_Quantity__c < 10'

);

}

global void execute(Database.BatchableContext BC, List<SObject> records) {

List<HandsMen_Product__c> productsToUpdate = new List<HandsMen_Product__c>();

// Cast SObject list to Product__c list

for (SObject record : records) {

HandsMen_Product__c product = (HandsMen_Product__c) record;

product.Stock_Quantity__c += 50; // Restock logic

productsToUpdate.add(product);

}
```

```
28
29  if (!productsToUpdate.isEmpty()) {
30
31  try {
32
33  update productsToUpdate;
34
35  } catch (DmlException e) {
36
37  System.debug('Error updating inventory: ' + e.getMessage());
38
39  }
40
41  }
42
43  }
44
45  global void finish(Database.BatchableContext BC) {
46
47  System.debug('Inventory Sync Completed');
48
49  }
50
51  // Scheduler Method
52
53  global void execute(SchedulableContext SC) {
54
55  InventoryBatchJob batchJob = new InventoryBatchJob();
56
57  Database.executeBatch(batchJob, 200);
58
59  }
60  }
```

Figure 19 InventoryBatchJob Apex Class

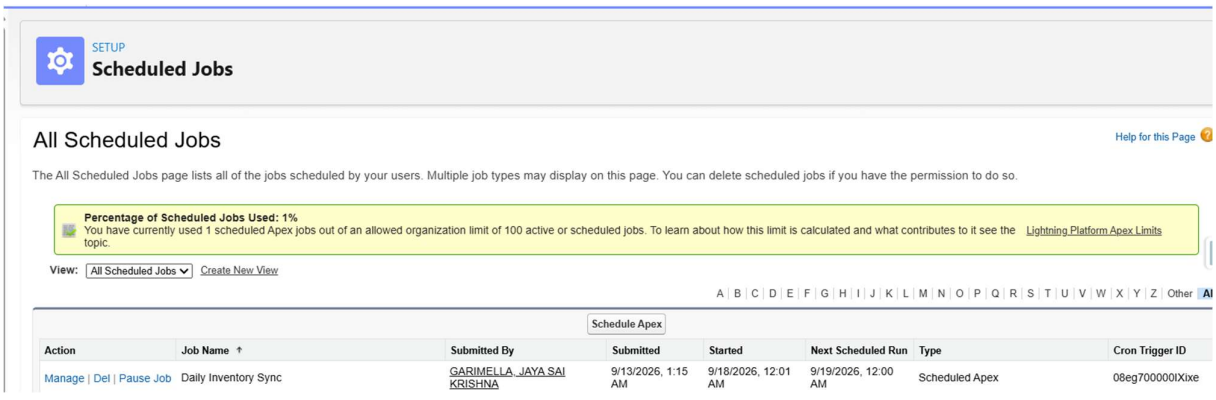
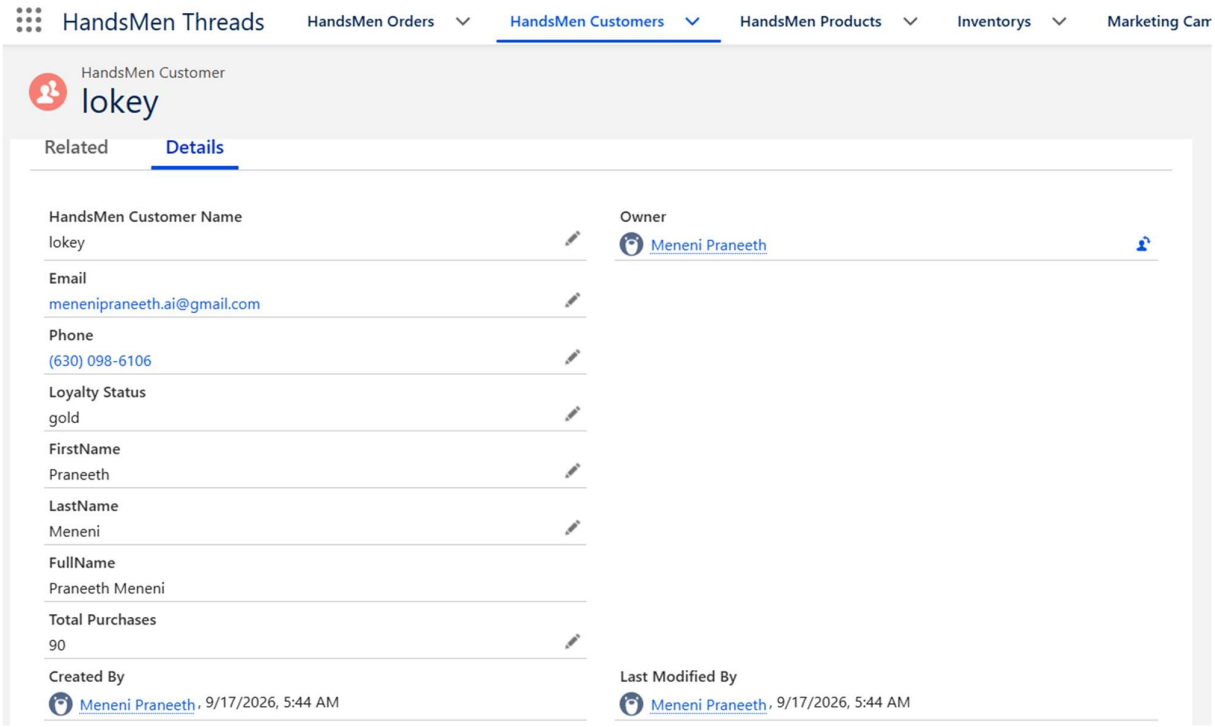


Figure 20 Scheduled Inventory Job

9 User Interface and Sample Records

Sample records should demonstrate that the configured fields, relationships, and automation are available to business users in Lightning Experience. Use realistic but non-sensitive data and keep the browser zoom between 80 and 100 percent so labels and values remain readable.

Record	Recommended Fields to Show
HandsMen Customer	Full name, email, phone, loyalty status, total purchases
HandsMen Product	Product name, SKU, price, stock quantity
HandsMen Order	Order number, customer, product, status, quantity, total amount
Inventory	Inventory number, product, warehouse, stock quantity, stock status
Marketing Campaign	Campaign number or name, dates, related customer



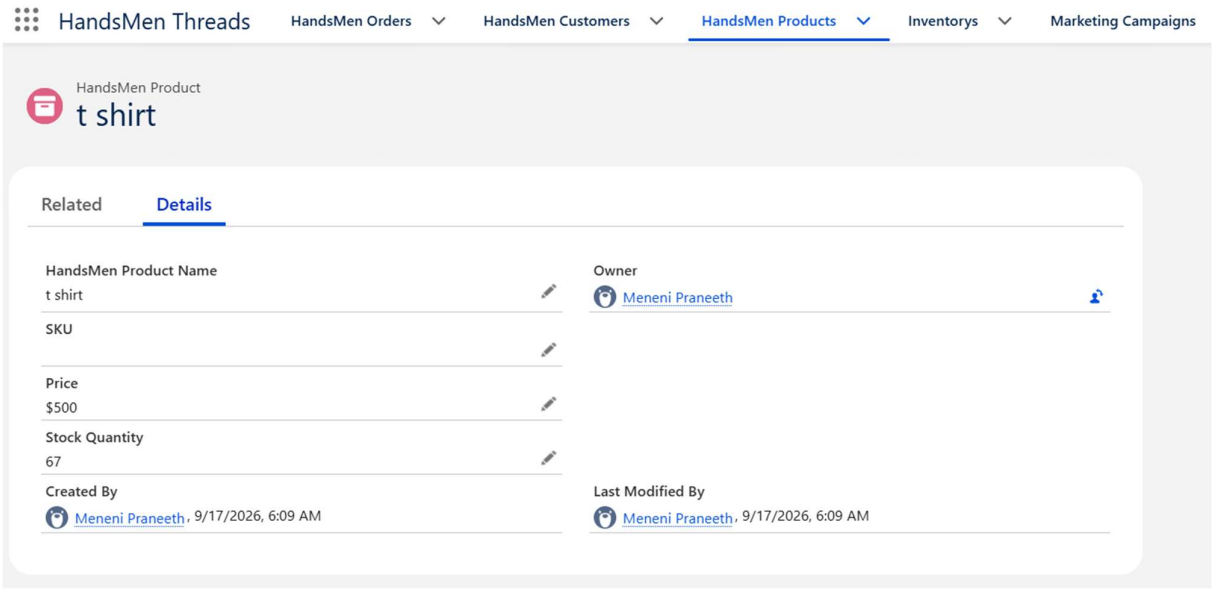


Figure 21 Customer and Product Sample Records

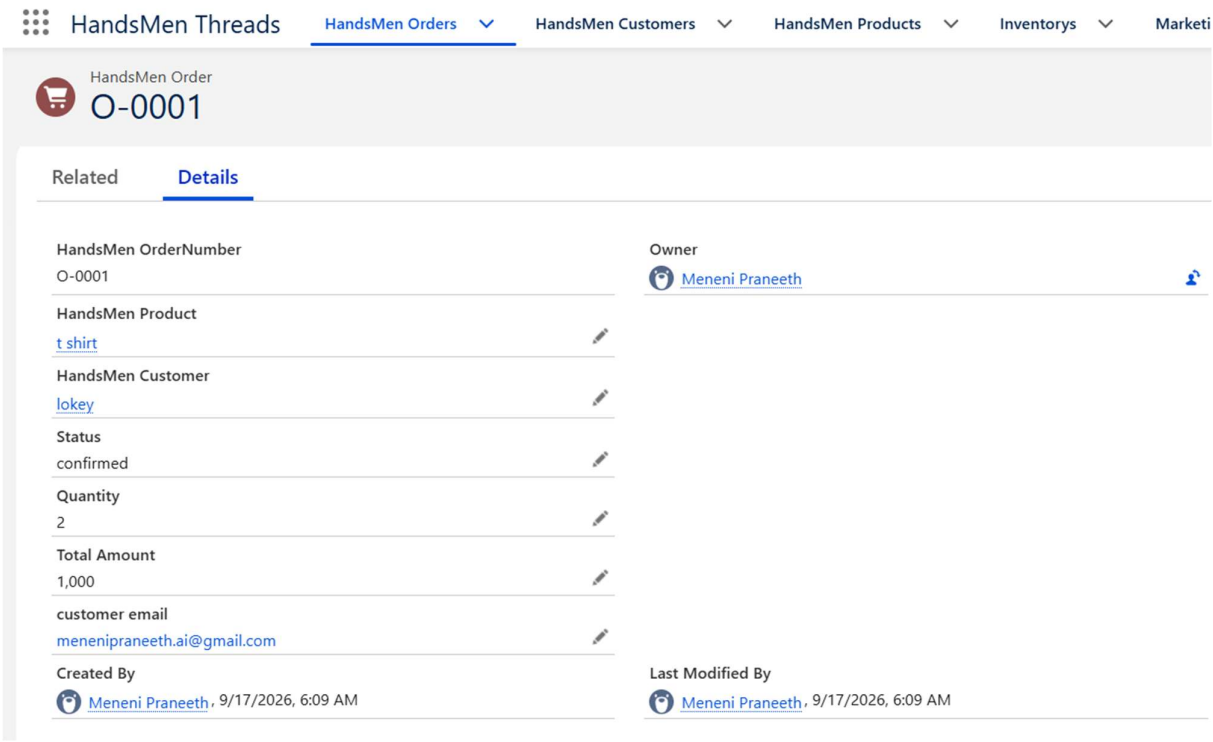


Figure 22 Order and Inventory Sample Records

10 Functional Testing Evidence

Functional testing should show both configuration and business outcomes. Capture the action and its result, and use before and after screenshots where a value changes.

Test Case	Action	Expected Result	Evidence
Order quantity validation	Save a status and quantity combination that violates the handler rule.	Save is blocked with the Apex error.	Error on Order page
Order total	Create or edit an order with product and quantity.	Total Amount equals Quantity multiplied by Price.	Order values
Stock deduction	Change an order from Pending to Confirmed.	Inventory decreases once by the ordered quantity.	Before and after stock
Order confirmation	Confirm an order.	Customer receives the confirmation email.	Email inbox
Low stock alert	Set stock below 5.	Inventory manager receives an alert.	Email inbox
Loyalty update	Run or debug the scheduled flow with purchase values.	Customer loyalty changes to Gold, Silver, or Bronze.	Flow debug and record
Batch processing	Execute InventoryBatchJob.	Low stock records are processed and the job completes.	Apex Jobs and record

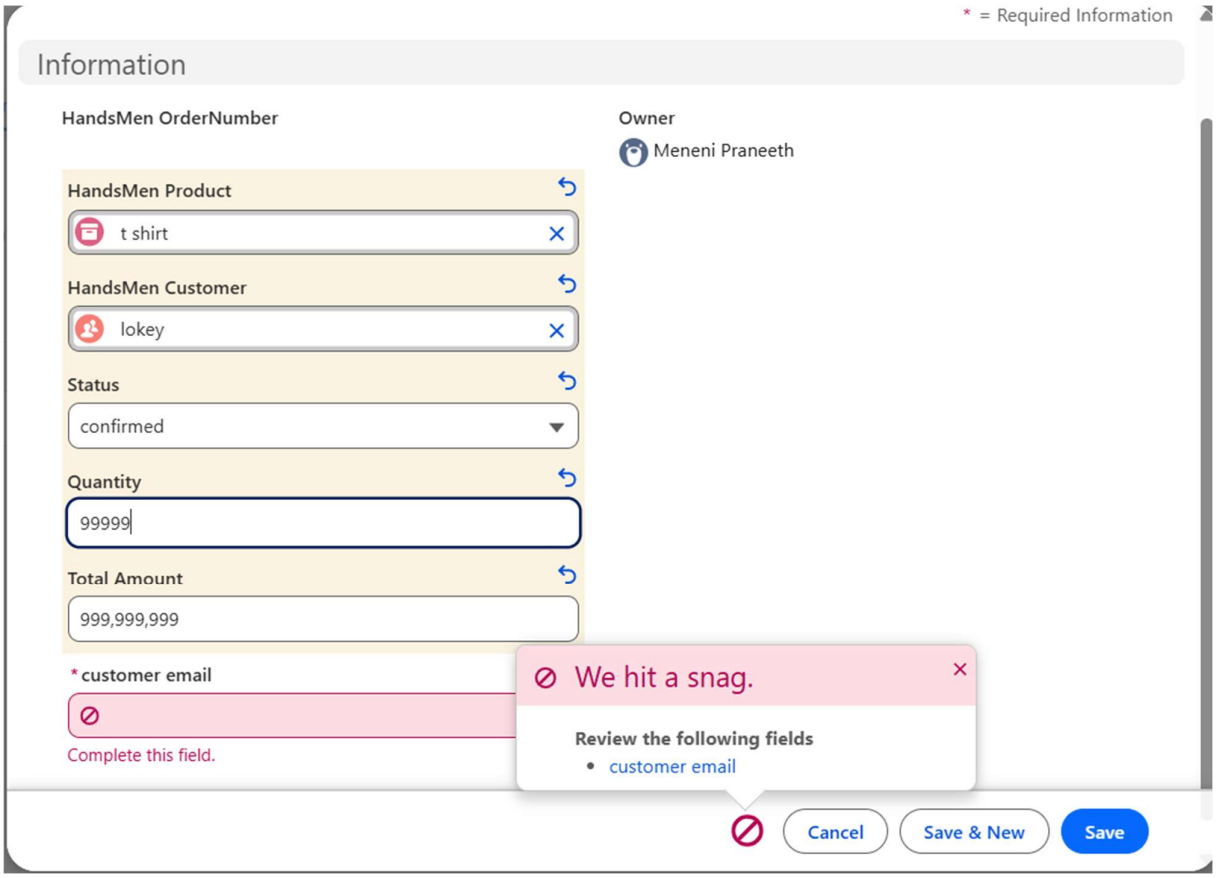


Figure 23 Order Validation Error

10.1 Automation Test Results

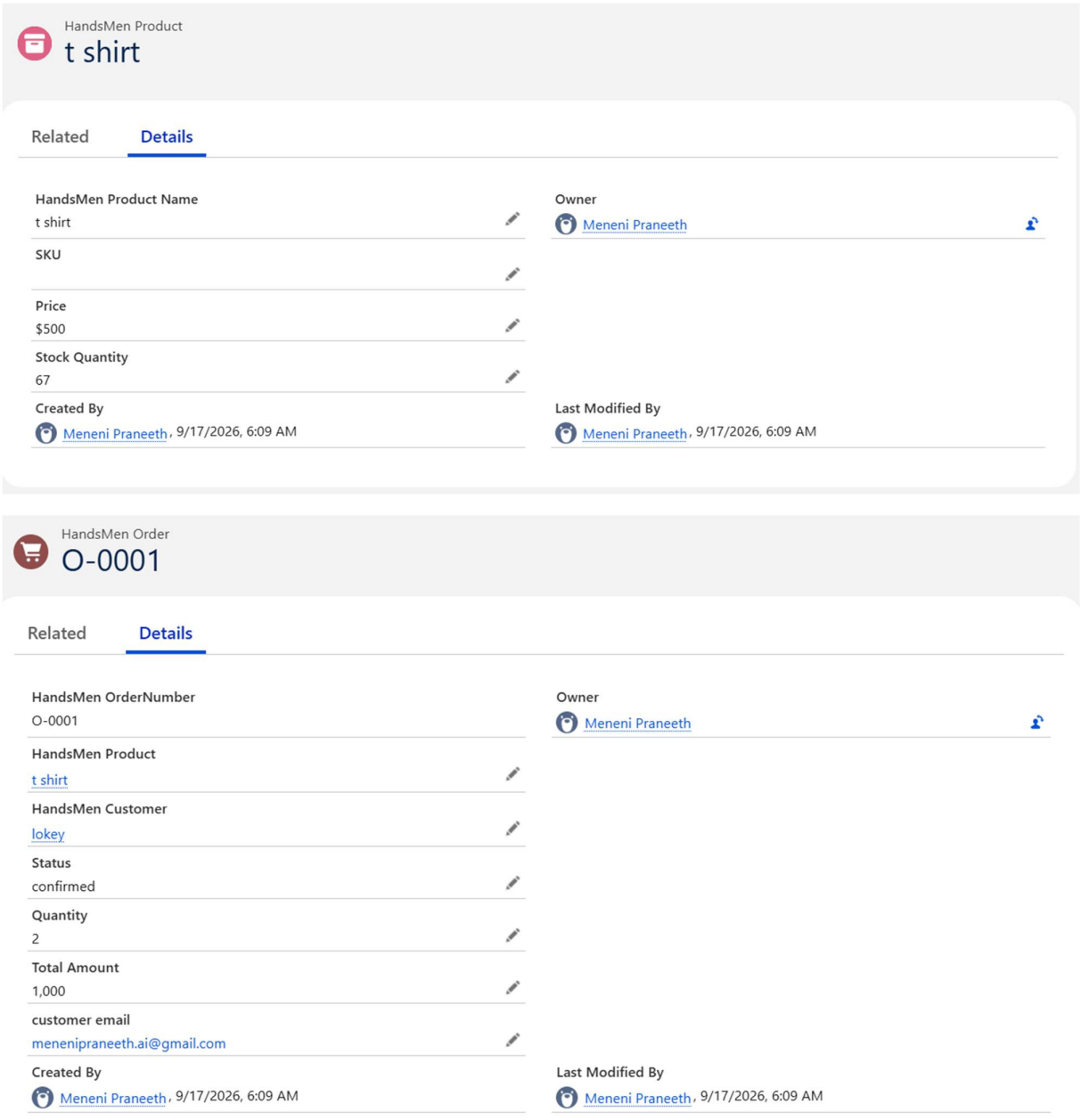


Figure 24 Order Total Calculation

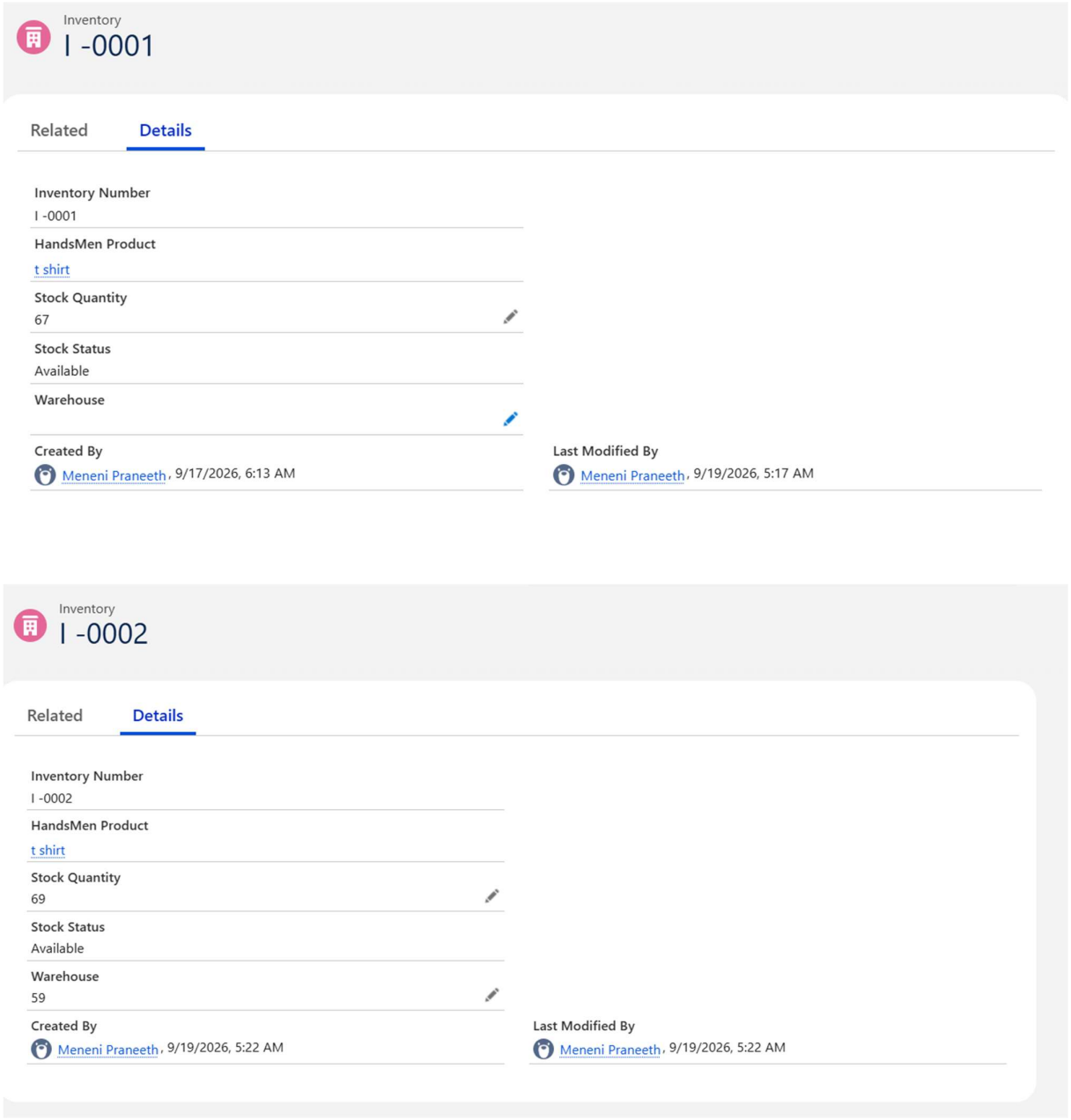
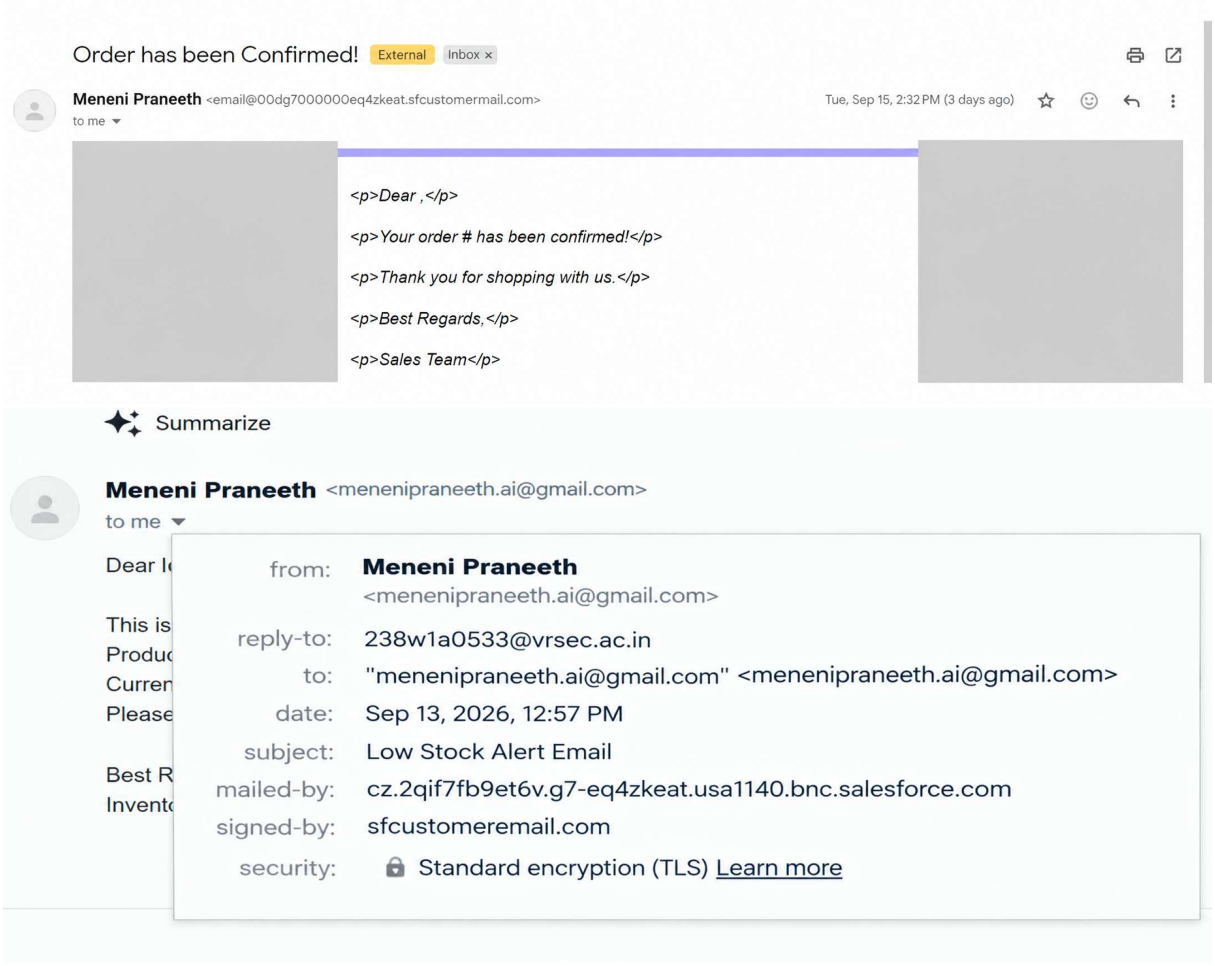


Figure 25 Inventory Before and After Confirmation



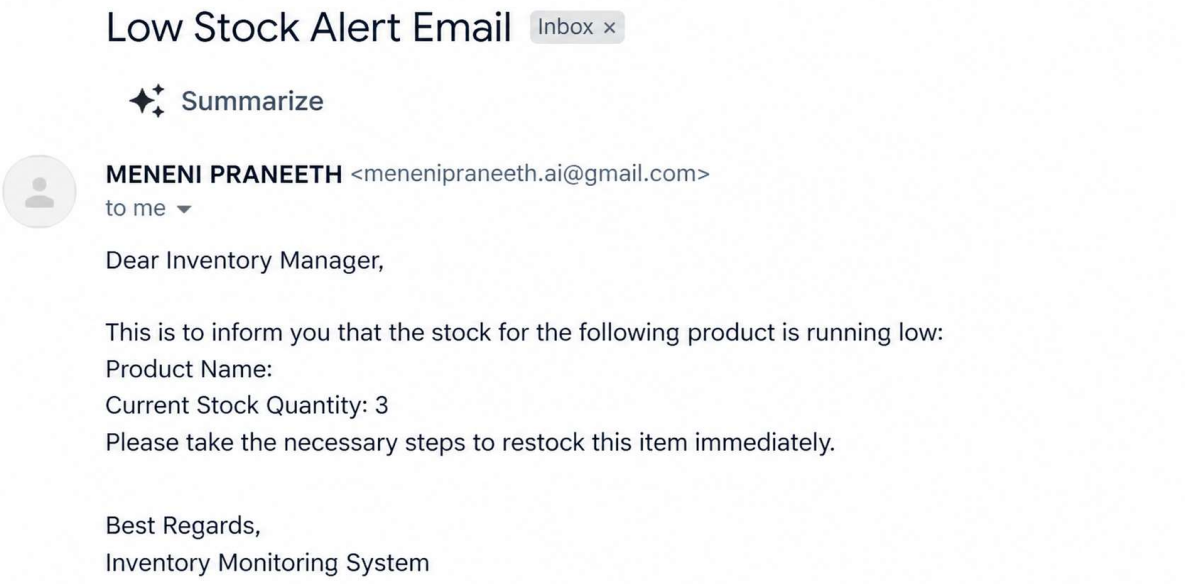


Figure 26 Automated Email Evidence

10.2 Scheduled Automation and Batch Results

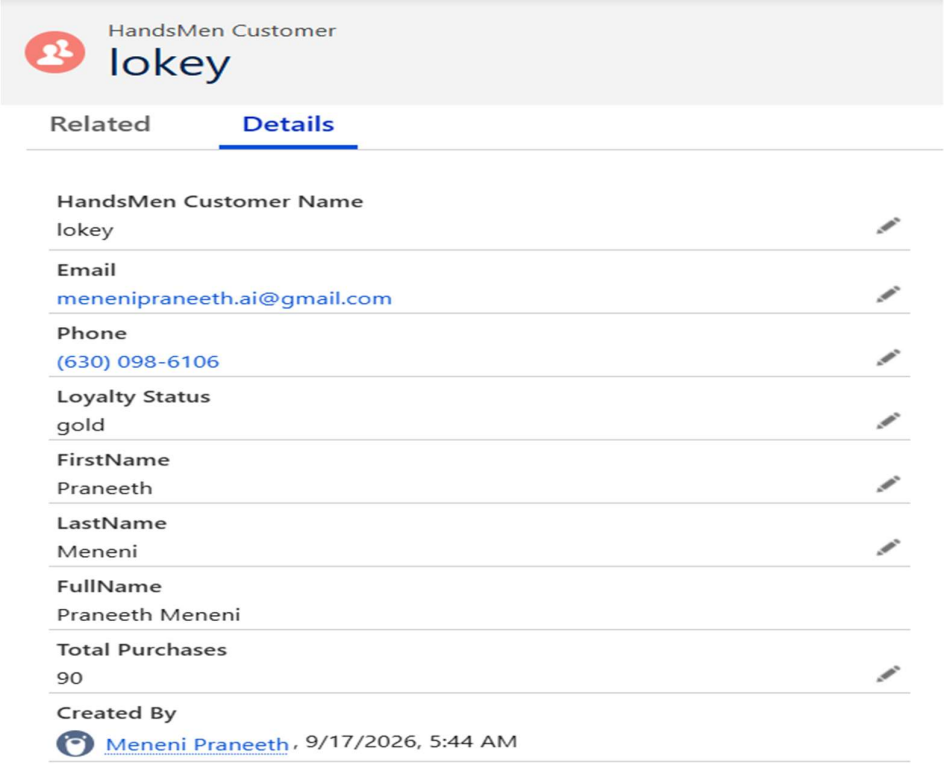


Figure 27 Loyalty Status Update Result

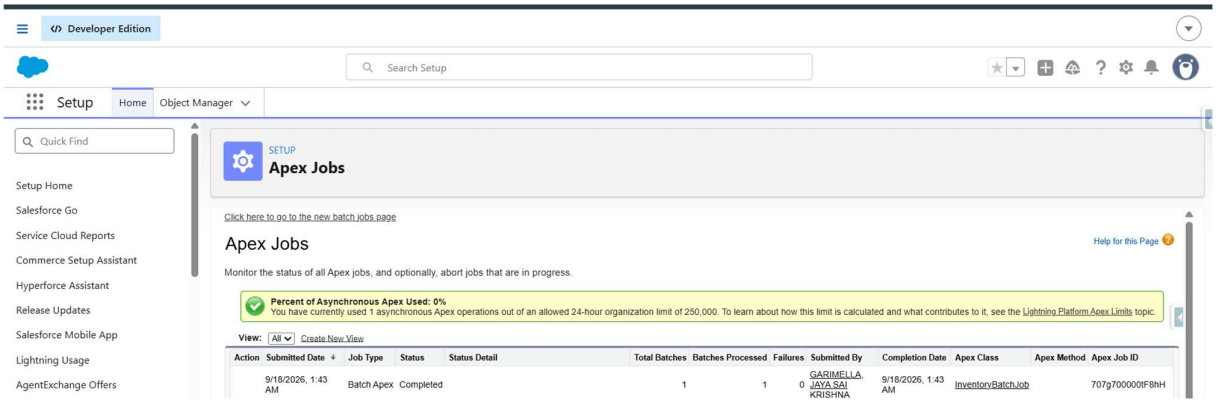


Figure 28 Batch Job Execution Result

11 Conclusion

The HandsMen Threads Salesforce project provides a practical CRM and operations solution for a fashion business. It organizes customer, product, order, inventory, and campaign data; applies validation and formula logic; controls access with Salesforce security features; automates customer and stock communication; and uses Apex for business rules and bulk processing.

Together, the configuration, automation, development, and testing evidence demonstrate core Salesforce Platform Developer skills. The completed screenshot set will give the reviewer a clear path from the data model and security design to working automation and functional outcomes.